

UW PICO 5.09

File: job_listing_fg.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 32
#define MAX_ARGS 32
#define BUF_SIZE 1024

typedef enum { RUNNING, STOPPED } JobState;

typedef struct {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[BUF_SIZE];
    JobState state;
} Job;

Job job_table[MAX_JOBS];
int next_job_id = 1;
pid_t shell_pgid;
struct termios shell_tmodes;

// Trim whitespace from string ends
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Add a job entry to the job table
int add_job(pid_t pid, pid_t pgid, const char *command, JobState state) {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid == 0) {
            job_table[i].job_id = next_job_id++;
            job_table[i].pid = pid;
            job_table[i].pgid = pgid;
            strncpy(job_table[i].command, command, BUF_SIZE - 1);
            job_table[i].state = state;
            return job_table[i].job_id;
        }
    }
}
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

UW PICO 5.09

File: job_listing_fg.c

```
int next_job_id = 1;
pid_t shell_pgid;
struct termios shell_tmodes;

// Trim whitespace from string ends
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Add a job entry to the job table
int add_job(pid_t pid, pid_t pgid, const char *command, JobState state) {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid == 0) {
            job_table[i].job_id = next_job_id++;
            job_table[i].pid = pid;
            job_table[i].pgid = pgid;
            strncpy(job_table[i].command, command, BUF_SIZE - 1);
            job_table[i].state = state;
            return job_table[i].job_id;
        }
    }
    printf("Job table full!\n");
    return -1;
}

// Non-blocking poll to clear or update terminated/stopped processes
void update_job_status() {
    int status;
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0) {
            pid_t result = waitpid(job_table[i].pid, &status, WNOHANG | WUNTRACED | WCONTINUED);
            if (result > 0) {
                if (WIFEXITED(status) || WIFSIGNALED(status)) {
                    job_table[i].pid = 0; // Remove finished job from table
                } else if (WIFSTOPPED(status)) {
                    job_table[i].state = STOPPED;
                } else if (WIFCONTINUED(status)) {
                    job_table[i].state = RUNNING;
                }
            }
        }
    }
}
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

UW PICO 5.09

File: job_listing_fg.c

```

    }
}
printf("Job table full!\n");
return -1;
}

// Non-blocking poll to clear or update terminated/stopped processes
void update_job_status() {
    int status;
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0) {
            pid_t result = waitpid(job_table[i].pid, &status, WNOHANG | WUNTRACED | WCONTINUED);
            if (result > 0) {
                if (WIFEXITED(status) || WIFSIGNALED(status)) {
                    job_table[i].pid = 0; // Remove finished job from table
                } else if (WIFSTOPPED(status)) {
                    job_table[i].state = STOPPED;
                } else if (WIFCONTINUED(status)) {
                    job_table[i].state = RUNNING;
                }
            }
        }
    }
}

// List active jobs in formatted layout
void list_jobs() {
    update_job_status(); // Sync state before printing
    int active_jobs = 0;
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0) {
            active_jobs++;
            const char *state_str = (job_table[i].state == RUNNING) ? "Running" : "Stopped";
            printf("[%d] %-10s %s\n", job_table[i].job_id, state_str, job_table[i].command);
        }
    }
    if (!active_jobs) {
        printf("No active background or stopped jobs.\n");
    }
}

// Bring job to foreground and transfer terminal control
void bring_to_foreground(int job_id) {
    int job_idx = -1;
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0 && job_table[i].job_id == job_id) {
            job_idx = i;
            break;
        }
    }
}

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Text

UW PICO 5.09

File: job_listing_fg.c

```

// List active jobs in formatted layout
void list_jobs() {
    update_job_status(); // Sync state before printing
    int active_jobs = 0;
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0) {
            active_jobs++;
            const char *state_str = (job_table[i].state == RUNNING) ? "Running" : "Stopped";
            printf("[%d] %-10s %s\n", job_table[i].job_id, state_str, job_table[i].command);
        }
    }
    if (!active_jobs) {
        printf("No active background or stopped jobs.\n");
    }
}

// Bring job to foreground and transfer terminal control
void bring_to_foreground(int job_id) {
    int job_idx = -1;
    for (int i = 0; i < MAX_JOBS; i++) {
        if (job_table[i].pid > 0 && job_table[i].job_id == job_id) {
            job_idx = i;
            break;
        }
    }

    if (job_idx == -1) {
        printf("fg: job [%d] not found\n", job_id);
        return;
    }

    Job *j = &job_table[job_idx];
    printf("%s\n", j->command);

    // Give control of terminal to process group
    tcsetpgrp(STDIN_FILENO, j->pgid);

    // Send SIGCONT if stopped
    if (j->state == STOPPED) {
        kill(-j->pgid, SIGCONT);
        j->state = RUNNING;
    }

    // Synchronously wait for target process to finish or stop
    int status;
    waitpid(j->pid, &status, WUNTRACED);
}

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Text

UW PICO 5.09

File: job_listing_fg.c

```

char *args[MAX_ARGS];
int arg_count = 0;
char *token = strtok(line, " \t");
while (token != NULL && arg_count < MAX_ARGS - 1) {
    args[arg_count++] = token;
    token = strtok(NULL, " \t");
}
args[arg_count] = NULL;

if (arg_count == 0) return;

// Built-in 'jobs' command
if (strcmp(args[0], "jobs") == 0) {
    list_jobs();
    return;
}

// Built-in 'fg' command
if (strcmp(args[0], "fg") == 0) {
    int target_jid = 1; // Default to job 1 if no argument provided
    if (arg_count > 1) {
        char *jid_str = args[1];
        if (jid_str[0] == '%') jid_str++;
        target_jid = atoi(jid_str);
    }
    bring_to_foreground(target_jid);
    return;
}

pid_t pid = fork();
if (pid < 0) {
    perror("fork failed");
    return;
} else if (pid == 0) {
    setpgid(0, 0);
    signal(SIGINT, SIG_DFL);
    signal(SIGTSTP, SIG_DFL);
    execvp(args[0], args);
    perror("Command execution failed");
    exit(EXIT_FAILURE);
} else {
    setpgid(pid, pid);
    if (is_bg) {
        int jid = add_job(pid, pid, original_cmd, RUNNING);
        printf("[%d] %d\n", jid, pid);
    } else {
        // Transfer terminal control for foreground job

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Text

UW PICO 5.09

File: job_listing_fg.c

```

    tcsetpgrp(STDIN_FILENO, pid);
    int status;
    waitpid(pid, &status, WUNTRACED);

    // Reclaim terminal control back to shell
    tcsetpgrp(STDIN_FILENO, shell_pgid);

    if (WIFSTOPPED(status)) {
        int jid = add_job(pid, pid, original_cmd, STOPPED);
        printf("\n[%d]+ Stopped\t\t%s\n", jid, original_cmd);
    }
}
}

int main() {
    char input_buf[BUF_SIZE];
    memset(job_table, 0, sizeof(job_table));

    // Ensure interactive shell controls terminal session
    shell_pgid = getpid();
    setpgid(shell_pgid, shell_pgid);
    tcsetpgrp(STDIN_FILENO, shell_pgid);

    // Ignore job control signals in parent shell
    signal(SIGINT, SIG_IGN);
    signal(SIGTSTP, SIG_IGN);

    while (1) {
        printf("JobFG Shell > ");
        fflush(stdout);

        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline
        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_command(input_buf);
    }

    return 0;
}

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Text

```
medikondarakesh@Medikondas-MacBook-Air ~ % nano job_listing_fg.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall job_listing_fg.c -o job_listing_fg
./job_listing_fg
_ _ _ _ _
```